

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего
образования «Санкт-Петербургский политехнический университет Петра
Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

«Основы баз данных»

Отчет о выполнении курсовой работы

**«Учет заказов ателье индивидуального
пошива»**

Студент,
группы 5130201/40003 _____ Лебедев К. О.

Руководитель,
преподаватель _____ Попов С.Г.

«____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

1	Проектирование	3
1.1	Конечная схема базы данных	6
1.2	Создание базы данных и заполнение значениями	9
2	Программирование	10
3	Задание 1. Работа с представлениями(VIEW)	10
3.1	Запрос №1.1	10
3.2	Запрос №1.2	11
3.3	Запрос №1.3	12
4	Триггеры	14
4.1	Постановка задачи	14
4.2	Создание постоянной таблицы	17
4.3	Функции для таблицы портных	17
4.4	Функции для таблицы администраторов	18
4.5	Функции для таблицы ателье	20
4.6	Создание триггеров	20
4.7	Тестовые примеры	21
4.7.1	Тестовые примеры для таблицы портных	22
4.7.2	Тестовые примеры для таблицы администраторов	23
4.7.3	Тестовые примеры для таблицы ателье	24

1 Проектирование

Таблица 1: Ателье - Atelier

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Ателье_id	Atelier_id	INT	NOT NULL	PK	
Адрес	Address	VARCHAR(200)	NOT NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		

Таблица 2: Заказчик - Client

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Заказчик_id	Client_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		

Таблица 3: Ателье_Заказчик - Atelier_Client

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Заказчик_id	Client_id	INT	NOT NULL	FK	Client
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 4: Администратор - Administrator

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Админ-р_id	Admin_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NOT NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 5: Портной - Tailor

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Портной_id	Tailor_id	INT	NOT NULL	PK	
Фамилия	Surname	VARCHAR(40)	NOT NULL		
Имя	Name	VARCHAR(15)	NOT NULL		
Отчество	Patronymic	VARCHAR(20)	NULL		
Телефон	Phone	VARCHAR(15)	NOT NULL		
Эл_почта	Email	VARCHAR(320)	NULL		
Дата_рождения	Birth_date	DATE	NOT NULL		
Ателье_id	Atelier_id	INT	NOT NULL	FK	Atelier

Таблица 6: Заказ - Order

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Заказ_id	Order_id	INT	NOT NULL	PK	
Дата_оформ-я	Order_date	DATE	NOT NULL		
Дата_окон-я	Completion_date	DATE	NULL		
Статус	Status	ENUM	'Принят'		
Общ._стоимость	Total_cost	DECIMAL(10,2)	0.00		
Заказчик_id	Client_id	INT	NOT NULL	FK	Client
Портной_id	Tailor_id	INT	NOT NULL	FK	Tailor
Админ-р_id	Admin_id	INT	NOT NULL	FK	Admin

Таблица 7: Изделие - Garment

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Изделие_id	Garment_id	INT	NOT NULL	PK	
Тип_изделия	Product_type	ENUM	NOT NULL		
Крой	Cut_style	ENUM	NOT NULL		
Заказ_id	Order_id	INT	NOT NULL	FK	Order
Портной_id	Tailor_id	INT	NOT NULL	FK	Tailor

Таблица 8: Ткань - Fabric

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Ткань_id	Fabric_id	INT	NOT NULL	PK	
Артикул	Article	VARCHAR(20)	NOT NULL		
Производитель	Manufacturer	VARCHAR(60)	NOT NULL		
Состав	Composition	VARCHAR(75)	NOT NULL		
Метраж	Meters_used	DECIMAL(10,2)	NOT NULL		
Цвет	Color	VARCHAR(25)	NOT NULL		
Применимость	Applicability	ENUM	'Основная'		

Таблица 9: Ткань_Изделие - Fabric_Garment

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Изделие_id	Garment_id	INT	NOT NULL	FK	Garment
Ткань_id	Fabric_id	INT	NOT NULL	FK	Fabric

Таблица 10: Фурнитура - Accessories

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Фурнитура_id	Accessory_id	INT	NOT NULL	PK	
Тип	Type	VARCHAR(30)	NOT NULL		
Материал	Material	VARCHAR(40)	NOT NULL		
Производитель	Manufacturer	VARCHAR(60)	NOT NULL		
Артикул	Article	VARCHAR(20)	NOT NULL		
Количество	Quantity_used	INT	NOT NULL		

Таблица 11: Фурнитура_Изделие - Accessories_Garment

Атрибут (рус)	Атрибут (eng)	Тип	По умолчанию	Ключ	Ссылка
Изделие_id	Garment_id	INT	NOT NULL	FK	Garment
Фурнитура_id	Accessory_id	INT	NOT NULL	FK	Accessories

1.1 Конечная схема базы данных

Далее на Рис. 3 и Рис. 4 представлены конечные схемы базы данных на русском и английском языке соответственно.

2

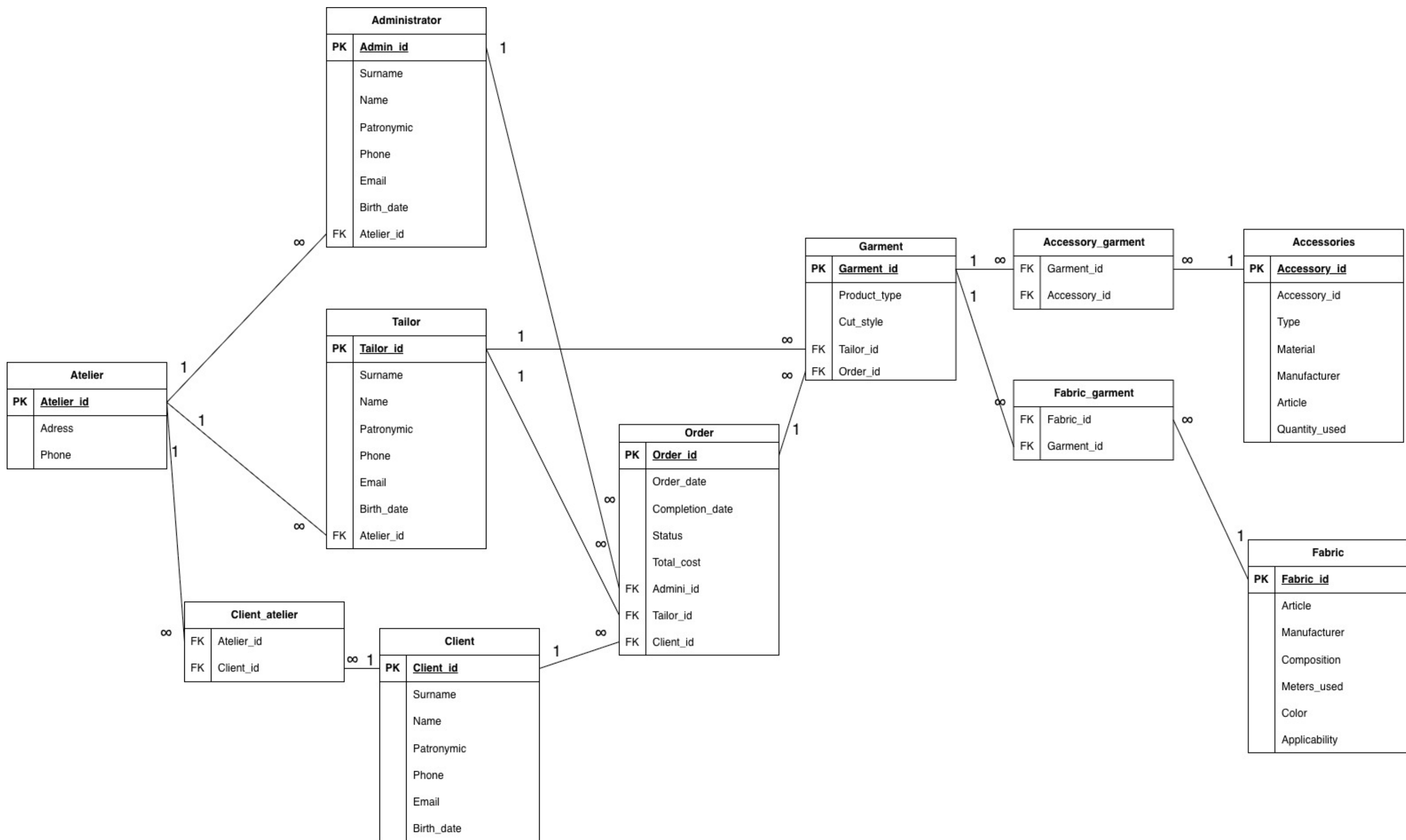


Рис. 2: Схема базы данных на английском.

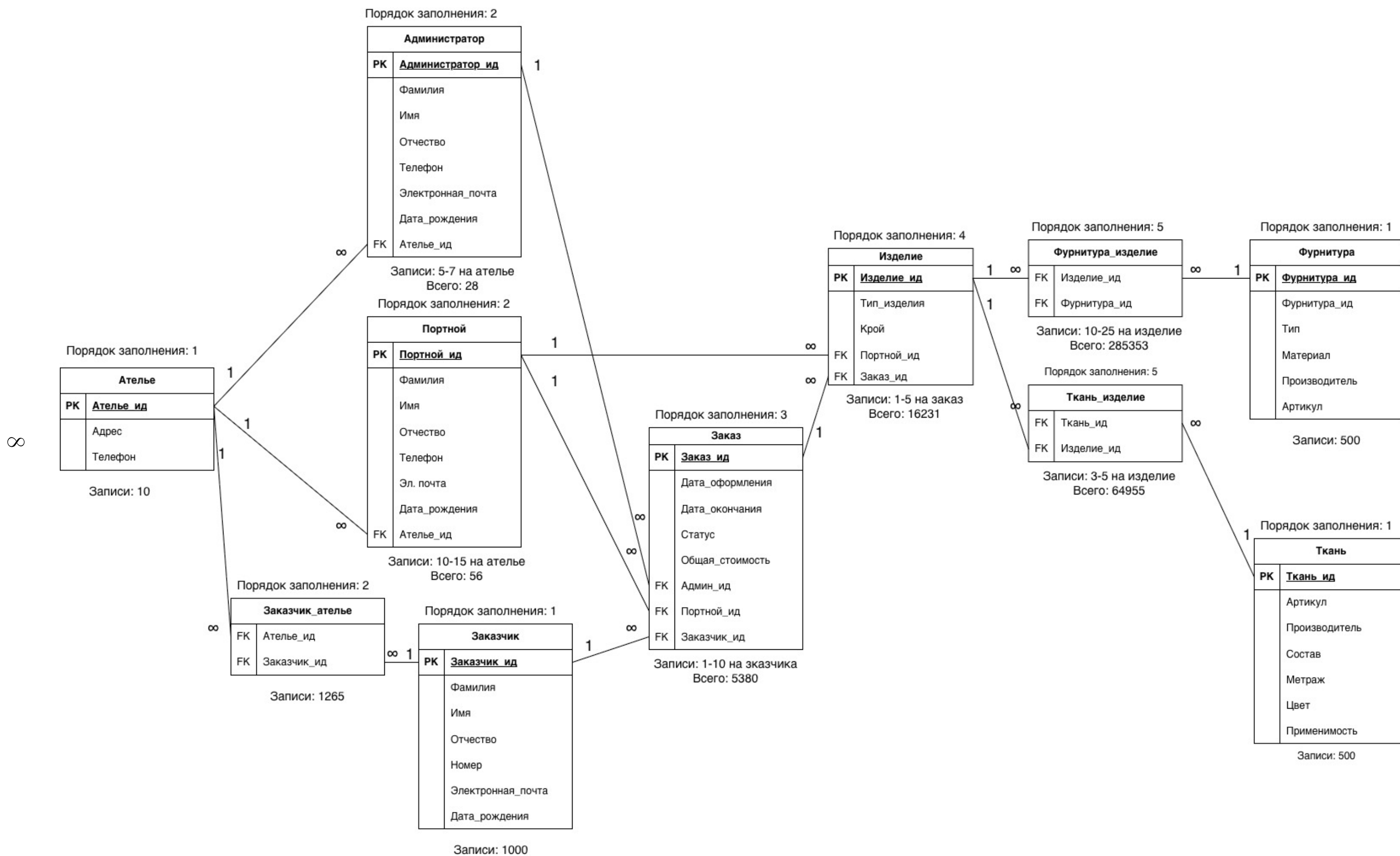


Рис. 1: Схема базы данных на русском.

1.2 Создание базы данных и заполнение значениями

База данных и таблицы, содержащиеся в ней, были созданы с помощью SQL-запросов и командной строки PostgreSQL. Для обеспечения целостности данных были настроены внешние ключи с каскадным удалением, а также созданы составные первичные ключи для таблиц-связок. Код создания базы данных представлен в приложении А.

База данных была заполнена случайными значениями, сгенерированными программой на языке C++. Для подключения к базе данных использовалась библиотека `libpq` — официальный клиентский интерфейс PostgreSQL. Заполнение выполнялось последовательно: сначала заполнялись независимые таблицы-словари, затем таблицы, содержащие внешние ключи. Всего было выполнено 9 этапов заполнения для 11 таблиц. В результате база данных содержит следующее количество записей, представленное в таблице 12.

Таблица 12: Количество записей в таблицах базы данных

Таблица	Количество записей
Ателье (atelier)	10
Портные (tailor)	123
Администраторы (administrator)	62
Заказчики (client)	3 000
Ателье_Заказчик (atelier_client)	3 831
Заказы (order)	16 499
Изделия (garment)	49 644
Ткани (fabric)	2 000
Фурнитура (accessories)	2 000
Ткань_Изделие (fabric_garment)	198 747
Фурнитура_Изделие (accessories_garment)	870 130
Итого	1 146 046

Код заполнения таблиц базы данных представлен в приложении Б.

2 Программирование

В данном разделе представлены SQL-запросы к спроектированной базе данных. Для каждого запроса приводится текстовое описание решаемой задачи, описание алгоритма выполнения, реляционная формула, листинг на языке SQL, результат выполнения.

3 Задание 1. Работа с представлениями(VIEW)

3.1 Запрос №1.1

Для каждого ателье посчитать количество портных и количество администраторов и создать соответствующее представление (VIEW).

Для подсчета количества сотрудников выполняется соединение таблицы `atelier` с таблицами `tailor` и `administrator` по внешнему ключу `atelier_id` с использованием `LEFT JOIN`. Группировка выполняется по идентификатору ателье. Для подсчета используется `COUNT(DISTINCT ...)`. Результат сортируется по идентификатору ателье.

Реляционная формула:

$$\text{atelier_employees_counts} = \pi \left(\begin{array}{c} \text{a.atelier_id}, \\ \gamma_{\text{count}(\text{t.tailor_id})(\text{tailor_count})}, \\ \gamma_{\text{count}(\text{ad.admin_id})(\text{admin_count})} \end{array} \right)$$

$$\begin{aligned} & \text{atelier } a \overset{LEFT}{\bowtie} (\text{t.atelier_id}=\text{a.atelier_id}) \text{ tailor } t \overset{LEFT}{\bowtie} (\text{ad.atelier_id}=\text{a.atelier_id}) \text{ administrator } ad \\ & \gamma_{\text{a.atelier_id}, \text{count}(\text{DISTINCT } t.\text{tailor_id}), \text{count}(\text{DISTINCT } ad.\text{admin_id})} \\ & \tau(\text{a.atelier_id}) \end{aligned}$$

```
1 CREATE VIEW atelier_employees_counts AS
2 SELECT
3     a.atelier_id,
4     COUNT(DISTINCT t.tailor_id) AS tailor_count,
5     COUNT(DISTINCT ad.admin_id) AS admin_count
6 FROM atelier a
7 LEFT JOIN tailor t ON t.atelier_id = a.atelier_id
8 LEFT JOIN administrator ad ON ad.atelier_id = a.atelier_id
9 GROUP BY a.atelier_id
10 ORDER BY a.atelier_id;
```

Листинг 1. Создание представления `atelier_employees_counts`

```
1 SELECT * FROM atelier_employees_counts;
```

Листинг 2. Запрос к представлению `atelier_employees_counts`

В результирующей таблице выведено 10 строк (по количеству ателье), время выполнения запроса составляет **5,725** мс. Результат выполнения запроса представлен на рисунке 5.

atelier_id	taylor_count	admin_count
1	12	6
2	13	5
3	13	6
4	11	5
5	12	6
6	10	7
7	12	7
8	14	7
9	11	6
10	15	7

(10 строк)

Рис. 3: Результат запроса №1.1

3.2 Запрос №1.2

Найти ателье, в которых число портных больше заданного значения N .

Запрос выполняется к созданному представлению `atelier_employees_counts`, которое содержит подсчитанное количество портных и администраторов для каждого ателье. С помощью параметризованного условия `WHERE taylor_count > N` производится фильтрация записей. В данном случае $N = 12$.

Реляционная формула:

$$\pi \left(\begin{array}{c} \text{atelier_id,} \\ \text{taylor_count as ,} \\ \text{admin_count} \end{array} \right) \\ \sigma(\text{taylor_count} > N)$$

```
1 SELECT
2     atelier_id,
3     taylor_count,
```

```

4      admin_count
5 FROM atelier_employees_counts
6 WHERE tailor_count > 12  -- N = 12;

```

Листинг 3. Запрос №1.2

В результирующей таблице выведено 4 строки (ателье, в которых количество портных превышает 12), время выполнения запроса составляет **24,466** мс. Далее на рисунке 6 представлен результат выполнения запроса.

atelier_id	tailor_count	admin_count
10	15	7
8	14	7
2	13	5
3	13	6

(4 строки)

Рис. 4: Результат запроса №1.2

На рисунке 7 представлена последовательность выполнения запроса.

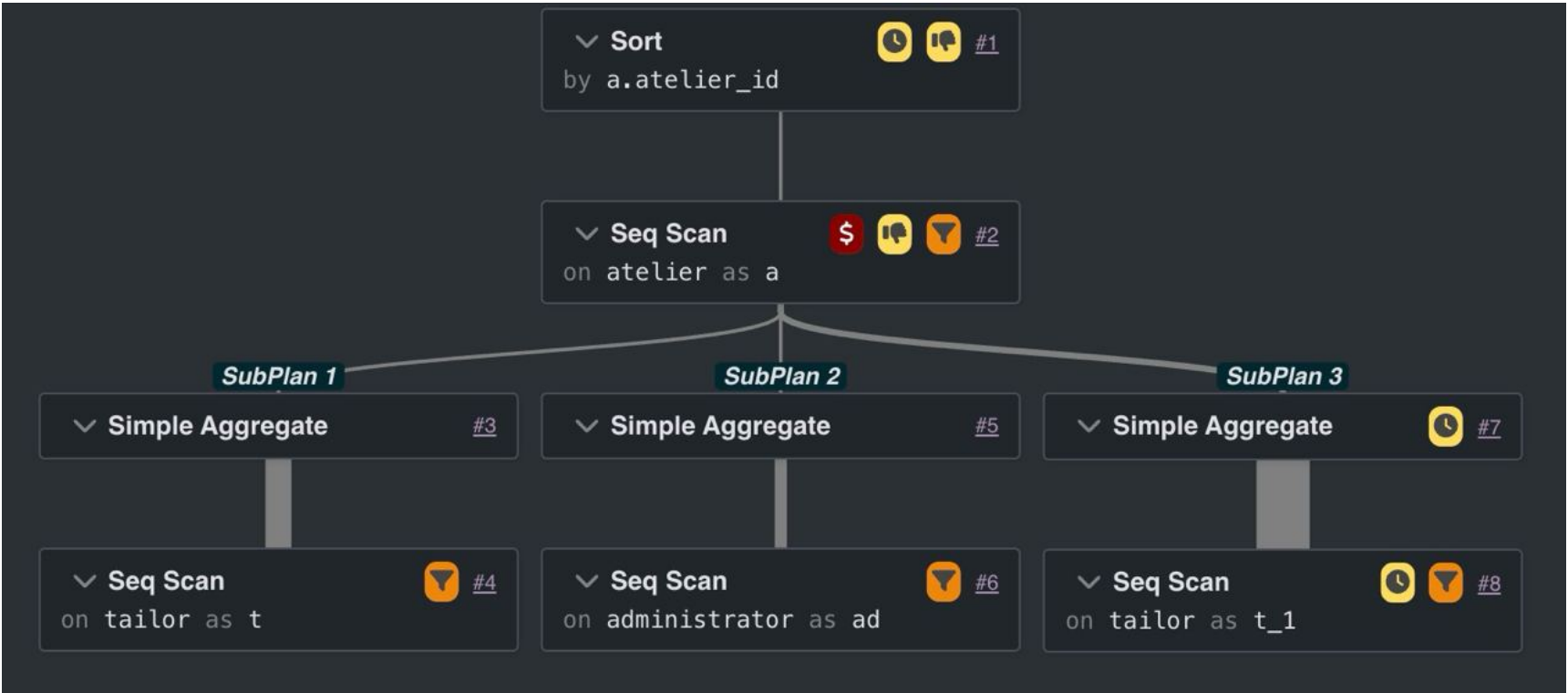


Рис. 5: Последовательность выполнения запроса №1.2

3.3 Запрос №1.3

Найти количество портных в ателье с числом завершенных заказов больше заданного значения N, по адресу в городе M.

Запрос выполняется с использованием представления `atelier_employees_counts`, которое содержит количество портных для каждого ателье. Для подсчета завершенных заказов выполняется соединение таблиц `atelier`, `tailor` и `"order"` по

внешним ключам. С помощью условия WHERE address LIKE '%M%' производится фильтрация по городу (в данном случае M = Санкт-Петербург), а условие WHERE status IN ('Готов', 'Выдан') отбирает только завершённые заказы. Группировка выполняется по atelier_id с использованием HAVING COUNT(DISTINCT o.order_id) > N для фильтрации ателье с более чем N завершёнными заказами (в данном случае N = 600).

Реляционная формула:

$$\pi \left(\begin{array}{c} a.atelier_id, \\ a.address, \\ aec.tailor_count \end{array} \right)$$

$$\sigma(a.address \sim \langle M \rangle \wedge o.status \in \{ \langle \text{Готов} \rangle, \langle \text{Выдан} \rangle \})$$

$$atelier_employees_counts \Join_{(a.atelier_id=aec.atelier_id)} atelier \Join_{(t.atelier_id=a.atelier_id)} tailor \Join_{(o.tailor_id=t.tailor_id)} "order" \Join_{\gamma a.atelier_id, a.address, aec.tailor_count, count(o.order_id)} o$$

$$\sigma(count(o.order_id) > N)$$

```

1 SELECT
2     a.atelier_id,
3     a.address,
4     aec.tailor_count AS total_tailors
5 FROM atelier_employees_counts aec
6 JOIN atelier a ON a.atelier_id = aec.atelier_id
7 JOIN tailor t ON t.atelier_id = a.atelier_id
8 JOIN "order" o ON o.tailor_id = t.tailor_id
9 WHERE a.address LIKE '%Санкт-Петербург%'
10    AND o.status IN ('Готов', 'Выдан')
11 GROUP BY a.atelier_id, a.address, aec.tailor_count
12 HAVING COUNT(DISTINCT o.order_id) > 600;
```

Листинг 4. Запрос №1.3

В результирующей таблице выведена 1 строка (ателье в Санкт-Петербурге, имеющее более 600 завершённых заказов), время выполнения запроса составляет **19,617** мс. На рисунке 8 представлен результат выполнения запроса.

atelier_id	address	total_tailors
3	Санкт-Петербург, Невский пр., д. 47	13
(1 строка)		

Рис. 6: Результат запроса №1.3

Далее на рисунке 9 представлена последовательность выполнения запроса.

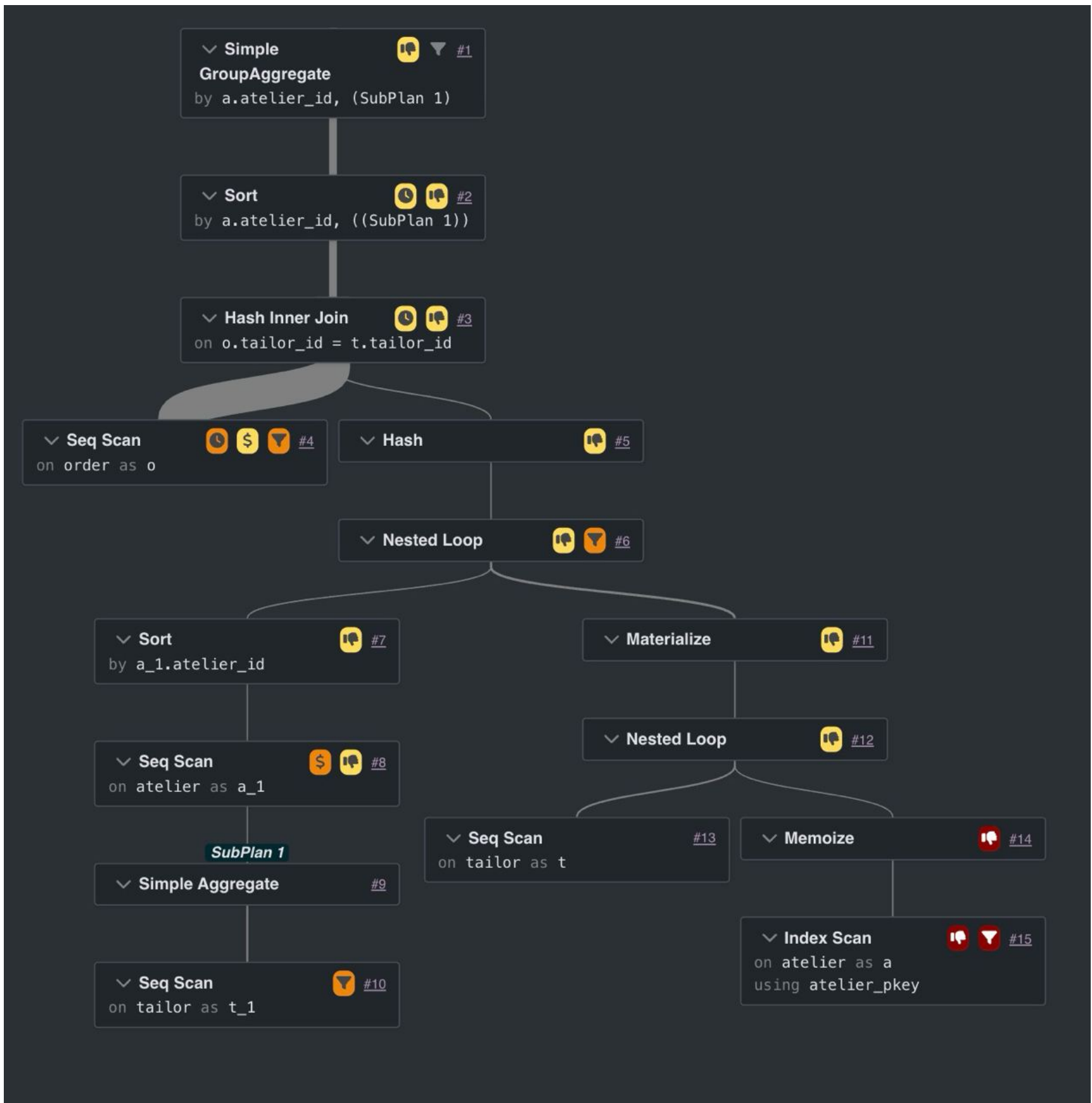


Рис. 7: Последовательность выполнения запроса №1.3

4 Триггеры

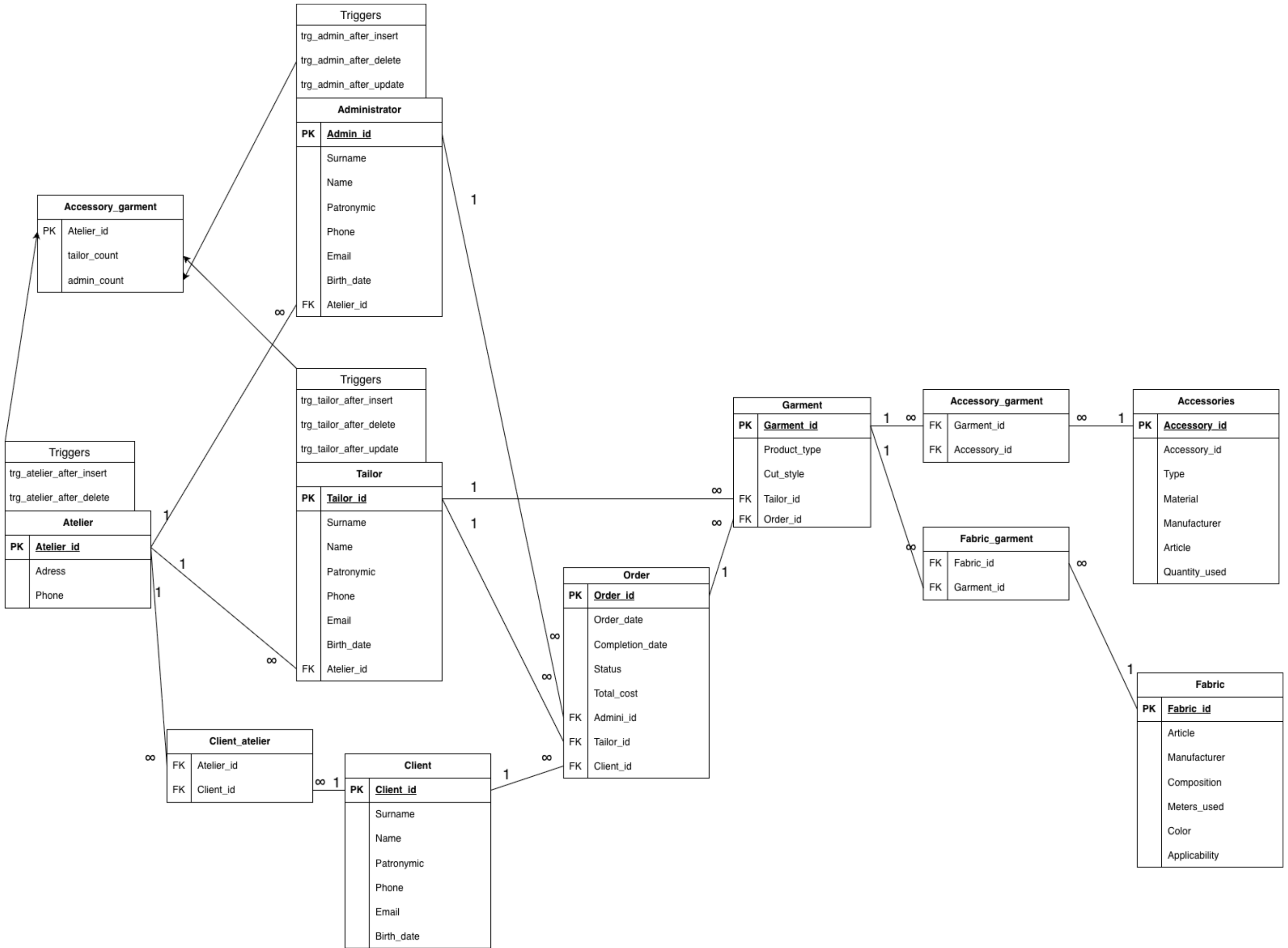
4.1 Постановка задачи

Требуется заменить представление `atelier_employees_counts` на постоянную таблицу с аналогичной структурой и обеспечить её непрерывную актуальность с помощью триггеров. Триггеры должны срабатывать при любых операциях модификации данных (добавление, удаление, изменение), влияющих на агрегированные значения.

Значения `tailor_count` и `admin_count` зависят только от таблиц `tailor` и `administrator`. Изменения в остальных таблицах (`client`, `"order"`, `garment`, `fabric`, `accessories` и связующих) на эти счётчики не влияют, поэтому триггеры создаются только на три таблицы: `atelier`, `tailor` и `administrator`. До-

полнительно необходимо автоматически создавать строку агрегата при появлении нового ателье и удалять её при удалении ателье.

Схема базы данных с указанием таблиц, триггеров и направления их действия приведена на рисунке 8.



4.2 Создание постоянной таблицы

Сначала удаляется представление и создаётся постоянная таблица с той же структурой. Поле `atelier_id` объявляется первичным ключом, что гарантирует одну строку на каждое ателье. Затем таблица однократно заполняется текущими значениями из таблиц `tailor` и `administrator`.

```
1 -- Удаление ранее созданного представления
2 DROP VIEW IF EXISTS atelier_employees_counts;
3
4 -- Создание постоянной таблицы
5 CREATE TABLE atelier_employees_counts (
6     atelier_id    INTEGER PRIMARY KEY,
7     tailor_count  INTEGER NOT NULL DEFAULT 0,
8     admin_count   INTEGER NOT NULL DEFAULT 0
9 );
10
11 -- Первичное заполнение таблицы
12 INSERT INTO atelier_employees_counts (atelier_id, tailor_count, admin_count)
13 SELECT
14     a.atelier_id,
15     (SELECT COUNT(*) FROM tailor t WHERE t.atelier_id = a.atelier_id),
16     (SELECT COUNT(*) FROM administrator ad WHERE ad.atelier_id = a.atelier_id)
17 FROM atelier a;
```

Листинг 4. Создание и заполнение таблицы `atelier_employees_counts`

4.3 Функции для таблицы портных

Для таблицы `tailor` создаются три функции: обработка добавления, удаления и перевода портного. Во всех случаях логика одинакова: значение `tailor_count` пересчитывается через `COUNT(*)` по актуальному состоянию таблицы `tailor`. Такой подход (пересчёт вместо инкремента) устойчив к повторным срабатываниям и гарантирует, что агрегат всегда соответствует реальным данным.

```
1 -- Добавление портного
2 CREATE OR REPLACE FUNCTION fn_tailor_insert()
3 RETURNS TRIGGER AS $$
4 BEGIN
5     SET tailor_count = (
6         SELECT COUNT(*) FROM tailor WHERE atelier_id = NEW.atelier_id
7     )
8     WHERE atelier_id = NEW.atelier_id;
9     RETURN NEW;
10 END;
```

```

11 $$ LANGUAGE plpgsql;
12
13 -- Удаление портного
14 CREATE OR REPLACE FUNCTION fn_tailor_delete()
15 RETURNS TRIGGER AS $$
16 BEGIN
17     UPDATE atelier_employees_counts
18     SET tailor_count = (
19         SELECT COUNT(*) FROM tailor WHERE atelier_id = OLD.atelier_id
20     )
21     WHERE atelier_id = OLD.atelier_id;
22     RETURN OLD;
23 END;
24 $$ LANGUAGE plpgsql;
25
26 -- Перевод портного
27 CREATE OR REPLACE FUNCTION fn_tailor_update()
28 RETURNS TRIGGER AS $$
29 BEGIN
30     UPDATE atelier_employees_counts
31     SET tailor_count = (
32         SELECT COUNT(*) FROM tailor WHERE atelier_id = OLD.atelier_id
33     )
34     WHERE atelier_id = OLD.atelier_id;
35
36     UPDATE atelier_employees_counts
37     SET tailor_count = (
38         SELECT COUNT(*) FROM tailor WHERE atelier_id = NEW.atelier_id
39     )
40     WHERE atelier_id = NEW.atelier_id;
41
42     RETURN NEW;
43 END;
44 $$ LANGUAGE plpgsql;

```

Листинг 5. Функции для таблицы tailor

4.4 Функции для таблицы администраторов

Для таблицы `administrator` создаются аналогичные три функции, отличающиеся только именем обновляемого столбца (`admin_count`). Логика полностью повторяет обработку портных.

```

1 -- Добавление администратора
2 CREATE OR REPLACE FUNCTION fn_admin_insert()
3 RETURNS TRIGGER AS $$

```

```

4 BEGIN
5     UPDATE atelier_employees_counts
6     SET admin_count = (
7         SELECT COUNT(*) FROM administrator WHERE atelier_id = NEW.atelier_id
8     )
9     WHERE atelier_id = NEW.atelier_id;
10    RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 -- Увольнение администратора
15 CREATE OR REPLACE FUNCTION fn_admin_delete()
16 RETURNS TRIGGER AS $$
17 BEGIN
18     UPDATE atelier_employees_counts
19     SET admin_count = (
20         SELECT COUNT(*) FROM administrator WHERE atelier_id = OLD.atelier_id
21     )
22     WHERE atelier_id = OLD.atelier_id;
23     RETURN OLD;
24 END;
25 $$ LANGUAGE plpgsql;
26
27 -- Перевод администратора
28 CREATE OR REPLACE FUNCTION fn_admin_update()
29 RETURNS TRIGGER AS $$
30 BEGIN
31     UPDATE atelier_employees_counts
32     SET admin_count = (
33         SELECT COUNT(*) FROM administrator WHERE atelier_id = OLD.atelier_id
34     )
35     WHERE atelier_id = OLD.atelier_id;
36
37     UPDATE atelier_employees_counts
38     SET admin_count = (
39         SELECT COUNT(*) FROM administrator WHERE atelier_id = NEW.atelier_id
40     )
41     WHERE atelier_id = NEW.atelier_id;
42
43     RETURN NEW;
44 END;
45 $$ LANGUAGE plpgsql;

```

Листинг 6. Функции для таблицы administrator

4.5 Функции для таблицы ателье

При появлении нового ателье необходимо сразу создать строку агрегата с нулевыми счётчиками, чтобы последующие операции вставки сотрудников могли её обновлять. При удалении ателье строка агрегата удаляется.

```
1 -- Добавление нового ателье
2 CREATE OR REPLACE FUNCTION fn_atelier_insert()
3 RETURNS TRIGGER AS $$
4 BEGIN
5     INSERT INTO atelier_employees_counts (atelier_id, tailor_count, admin_count)
6     VALUES (NEW.atelier_id, 0, 0);
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 -- Удаление ателье
12 CREATE OR REPLACE FUNCTION fn_atelier_delete()
13 RETURNS TRIGGER AS $$
14 BEGIN
15     DELETE FROM atelier_employees_counts WHERE atelier_id = OLD.atelier_id;
16     RETURN OLD;
17 END;
18 $$ LANGUAGE plpgsql;
```

Листинг 7. Функции для таблицы atelier

4.6 Создание триггеров

Каждая функция привязывается к соответствующей таблице и событию с помощью конструкции `CREATE TRIGGER ... AFTER ... FOR EACH ROW`. Выбор момента `AFTER` обусловлен тем, что агрегат должен вычисляться по уже изменённому состоянию таблицы: при `BEFORE` подзапрос `COUNT(*)` вернул бы устаревшее значение. Все триггеры строчные (`FOR EACH ROW`), что обеспечивает корректную работу как с одиночными, так и с массовыми операциями.

```
1 -- Триггеры для таблицы ателье
2 CREATE TRIGGER trg_atelier_after_insert
3 AFTER INSERT ON atelier
4 FOR EACH ROW EXECUTE FUNCTION fn_atelier_insert();
5
6 CREATE TRIGGER trg_atelier_after_delete
7 AFTER DELETE ON atelier
8 FOR EACH ROW EXECUTE FUNCTION fn_atelier_delete();
9
```

```

10 -- Триггеры для таблицы портных
11 CREATE TRIGGER trg_tailor_after_insert
12 AFTER INSERT ON tailor
13 FOR EACH ROW EXECUTE FUNCTION fn_tailor_insert();
14
15 CREATE TRIGGER trg_tailor_after_delete
16 AFTER DELETE ON tailor
17 FOR EACH ROW EXECUTE FUNCTION fn_tailor_delete();
18
19 CREATE TRIGGER trg_tailor_after_update
20 AFTER UPDATE ON tailor
21 FOR EACH ROW EXECUTE FUNCTION fn_tailor_update();
22
23 -- Триггеры для таблицы администраторов
24 CREATE TRIGGER trg_admin_after_insert
25 AFTER INSERT ON administrator
26 FOR EACH ROW EXECUTE FUNCTION fn_admin_insert();
27
28 CREATE TRIGGER trg_admin_after_delete
29 AFTER DELETE ON administrator
30 FOR EACH ROW EXECUTE FUNCTION fn_admin_delete();
31
32 CREATE TRIGGER trg_admin_after_update
33 AFTER UPDATE ON administrator
34 FOR EACH ROW EXECUTE FUNCTION fn_admin_update();

```

Листинг 8. Создание триггеров

Итого создано восемь триггеров: два для таблицы **atelier** (добавление и удаление ателье), три для **tailor** и три для **administrator**.

4.7 Тестовые примеры

Для проверки корректности работы триггеров сформированы тестовые примеры, покрывающие все варианты изменения данных исходных таблиц. Перечень тестовых примеров с описанием проверяемого состояния приведён в таблице 13.

Таблица 13: Перечень тестовых примеров

№	Описание проверяемого состояния
1	Добавление портного в ателье №1 (tailor_count +1)
2	Удаление портного из ателье №1 (tailor_count -1)
3	Перевод портного из ателье №1 в ателье №2 (-1 в старом, +1 в новом)
4	Добавление администратора в ателье №1 (admin_count +1)
5	Удаление администратора из ателье №1 (admin_count -1)
6	Перевод администратора из ателье №1 в ателье №2 (-1 в старом, +1 в новом)
7	Добавление нового ателье (строка с нулевыми счётчиками)
8	Удаление созданного ателье (строка агрегата удаляется)

4.7.1 Тестовые примеры для таблицы портных

Проверяется срабатывание триггеров `trg_tailor_after_insert`, `trg_tailor_after_delete` и `trg_tailor_after_update`.

```
1 -- Исходное состояние
2 SELECT * FROM atelier_employees_counts
3 WHERE atelier_id IN (1, 2) ORDER BY atelier_id;
4
5 -- Тест 1. Добавление портного в ателье 1
6 INSERT INTO tailor (surname, name, patronymic, phone, birth_date, atelier_id)
7 VALUES ('Тестов', 'Тест', 'Тестович', '+79001234567', '1990-01-01', 1);
8
9 SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
10
11 -- Тест 2. Удаление портного из ателье 1
12 DELETE FROM tailor WHERE surname = 'Тестов';
13
14 SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
15
16 -- Тест 3. Перевод портного из ателье №1 в ателье 2
```

```

17 UPDATE tailor SET atelier_id = 2
18 WHERE tailor_id = (SELECT MIN(tailor_id) FROM tailor WHERE atelier_id = 1);
19
20 SELECT * FROM atelier_employees_counts
21 WHERE atelier_id IN (1, 2) ORDER BY atelier_id;

```

Листинг 9. Тестовые примеры для таблицы tailor

Описание выполнения: Тест 1 добавляет новую строку в таблицу `tailor` — срабатывает триггер `trg_tailor_after_insert`, значение `tailor_count` для ателье №1 увеличивается на 1. Тест 2 удаляет добавленную строку — срабатывает `trg_tailor_after_delete`, значение возвращается к исходному. Тест 3 изменяет `atelier_id` у существующего портного — срабатывает `trg_tailor_after_update`, значение уменьшается в старом ателье и увеличивается в новом. Результат приведён на рисунке 9.

<pre>atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id IN (1, 2) ORDER BY atelier_id; atelier_id tailor_count admin_count ----- 1 12 6 2 13 5 (2 строки)</pre>			Исходное состояние
<pre>atelier=# INSERT INTO tailor (surname, name, patronymic, phone, birth_date, atelier_id) VALUES ('Тестов', 'Тест', 'Тестович', '+79001234567', '1990-01-01', 1); INSERT 0 1 atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id = 1; atelier_id tailor_count admin_count ----- 1 13 6 (1 строка)</pre>			
<pre>atelier=# DELETE FROM tailor WHERE surname = 'Тестов'; SELECT * FROM atelier_employees_counts WHERE atelier_id = 1; DELETE 1 atelier_id tailor_count admin_count ----- 1 12 6 (1 строка)</pre>			Тест 1
<pre>atelier=# UPDATE tailor SET atelier_id = 2 WHERE tailor_id = (SELECT MIN(tailor_id) FROM tailor WHERE atelier_id = 1); UPDATE 1 atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id IN (1, 2) ORDER BY atelier_id; atelier_id tailor_count admin_count ----- 1 11 6 2 14 5 (2 строки)</pre>			Тест 2
			Тест 3

Рис. 9: Результат проверки триггеров таблицы tailor

4.7.2 Тестовые примеры для таблицы администраторов

Проверяется срабатывание триггеров `trg_admin_after_insert`, `trg_admin_after_delete` и `trg_admin_after_update`. Логика полностью аналогична тестам для портных.

```

1 -- Тест 4. Добавление администратора в ателье 1
2 INSERT INTO administrator
3     (surname, name, patronymic, phone, email, birth_date, atelier_id)
4 VALUES ('Адинов', 'Админ', 'Адинович', '+79007654321',
5         'admin@test.ru', '1985-05-05', 1);
6

```

```

7 SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
8
9 -- Тест 5. Удаление администратора из ателье 1
10 DELETE FROM administrator WHERE surname = 'Админов';
11
12 SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
13
14 -- Тест 6. Перевод администратора из ателье 1 в ателье 2
15 UPDATE administrator SET atelier_id = 2
16 WHERE admin_id = (SELECT MIN(admin_id) FROM administrator WHERE atelier_id = 1);
17
18 SELECT * FROM atelier_employees_counts
19 WHERE atelier_id IN (1, 2) ORDER BY atelier_id;

```

Листинг 10. Тестовые примеры для таблицы administrator

Описание выполнения: тест 4 добавляет администратора в ателье №1 — срабатывает `trg_admin_after_insert`, `admin_count` увеличивается на 1. Тест 5 удаляет администратора — `trg_admin_after_delete` возвращает значение к исходному. Тест 6 изменяет `atelier_id` — `trg_admin_after_update` корректирует оба ателье. Результат приведён на рисунке 10.

```

atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id IN (1, 2) ORDER BY atelier_id;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          1 |          11 |           6
          2 |          14 |           5
(2 строки)

```

Исходное состояние

```

atelier=# INSERT INTO administrator (surname, name, patronymic, phone, email, birth_date, atelier_id)
VALUES ('Админов', 'Админ', 'Админович', '+79007654321', 'admin@test.ru', '1985-05-05', 1);
INSERT 0 1
atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          1 |          11 |           7
(1 строка)

```

Тест 4

```

atelier=# DELETE FROM administrator WHERE surname = 'Админов';
DELETE 1
atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id = 1;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          1 |          11 |           6
(1 строка)

```

Тест 5

```

atelier=# UPDATE administrator SET atelier_id = 2
WHERE admin_id = (SELECT MIN(admin_id) FROM administrator WHERE atelier_id = 1);
UPDATE 1
atelier=# SELECT * FROM atelier_employees_counts WHERE atelier_id IN (1, 2) ORDER BY atelier_id;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          1 |          11 |           5
          2 |          14 |           6
(2 строки)

```

Тест 6

Рис. 10: Результат проверки триггеров таблицы administrator

4.7.3 Тестовые примеры для таблицы ателье

Проверяется срабатывание триггеров `trg_atelier_after_insert` и `trg_atelier_aft`

```

1 -- Тест 7. Добавление нового ателье

```

```

2 INSERT INTO atelier (address, phone)
3 VALUES ('Тестовый город, ул. Тестовая, д. 1', '+70000000000');
4
5 SELECT * FROM atelier_employees_counts ORDER BY atelier_id DESC LIMIT 3;
6
7 -- Тест 8. Добавление портного в новое ателье
8 INSERT INTO tailor (surname, name, patronymic, phone, birth_date, atelier_id)
9 VALUES ('Тестов', 'Тест', 'Тестович', '+79001234567', '1990-01-01',
10         (SELECT MAX(atelier_id) FROM atelier));
11
12 SELECT * FROM atelier_employees_counts ORDER BY atelier_id DESC LIMIT 3;
13
14 -- Тест 10. Удаление сотрудников и ателье
15 DELETE FROM tailor WHERE atelier_id = (SELECT MAX(atelier_id) FROM
16     ↪ atelier);
17 DELETE FROM administrator WHERE atelier_id = (SELECT MAX(atelier_id) FROM
18     ↪ atelier);
19 DELETE FROM atelier WHERE atelier_id = (SELECT MAX(atelier_id) FROM
20     ↪ atelier);
21
22 SELECT * FROM atelier_employees_counts ORDER BY atelier_id DESC LIMIT 3;

```

Листинг 11. Тестовые примеры для таблицы atelier

Описание выполнения: тест 7 добавляет новое ателье — срабатывает `trg_atelier_after_insert`, создаётся строка агрегата с нулевыми счётчиками. Тест 8 удаляет созданное ателье — срабатывает `trg_atelier_after_delete`, строка агрегата исчезает. Результат приведён на рисунке 11.

```
atelier=# SELECT * FROM atelier_employees_counts ORDER BY atelier_id;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          1 |          11 |           5
          2 |          14 |           6
          3 |          13 |           6
          4 |          11 |           5
          5 |          12 |           6
          6 |          10 |           7
          7 |          12 |           7
          8 |          14 |           7
          9 |          11 |           6
         10 |          15 |           7
(10 строк)

atelier=# INSERT INTO atelier (address, phone)
VALUES ('Тестовый город, ул. Тестовая, д. 1', '+70000000000');
INSERT 0 1
atelier=# SELECT * FROM atelier_employees_counts ORDER BY atelier_id DESC LIMIT 3;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          11 |           0 |           0
          10 |          15 |           7
           9 |          11 |           6
(3 строки)

atelier=# DELETE FROM atelier WHERE atelier_id = (SELECT MAX(atelier_id) FROM atelier);
DELETE 1
atelier=# SELECT * FROM atelier_employees_counts ORDER BY atelier_id DESC LIMIT 3;
 atelier_id | tailor_count | admin_count
-----+-----+-----
          10 |          15 |           7
           9 |          11 |           6
           8 |          14 |           7
(3 строки)
```

Исходное состояние

Тест 7

Тест 8

Рис. 11: Результат проверки триггеров таблицы atelier